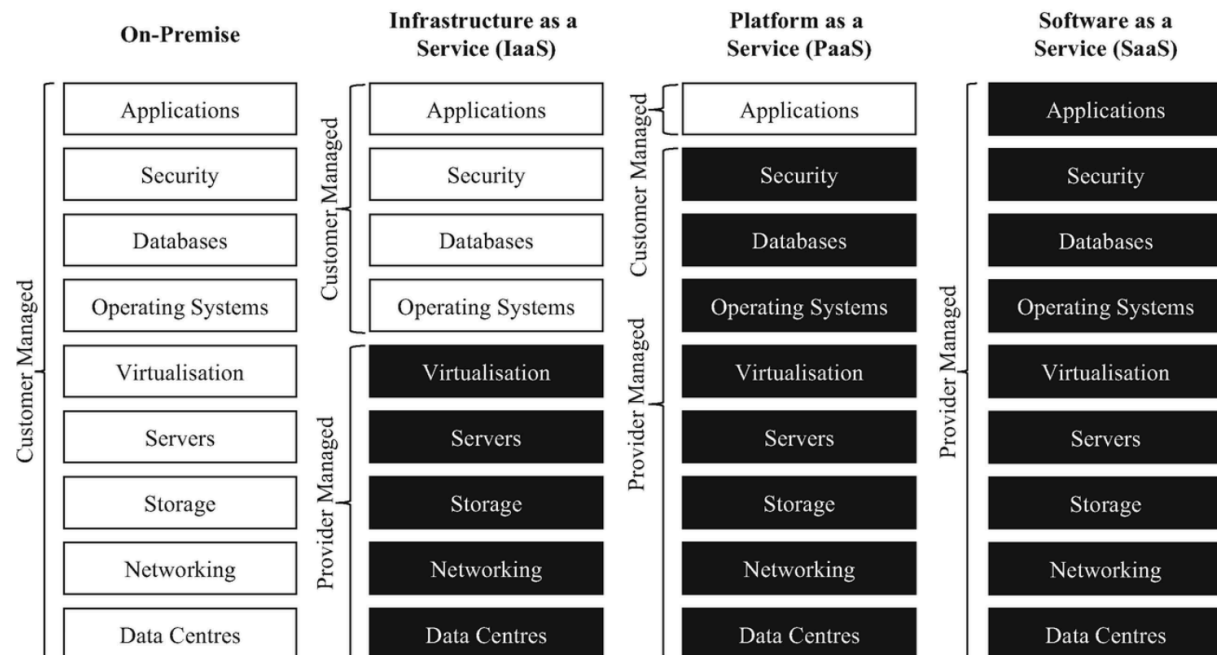


SSL Admin Task Induction

14-03-2026

Azure - cloud computing platform by microsoft

Cloud Computing - a paradigm for enabling network access to a scalable and elastic pool of shareable physical or virtual resources with self-service provisioning and administration on demand.



Ubuntu - published on a six-month release cycle, with long term support (LTS) versions issued every two years.

15-03-2026

SSH **Secure Shell Protocol (SSH Protocol)** is a [cryptographic network protocol](#) for operating [network services](#) securely over an [unsecured network](#). Its most notable applications are [remote login](#) and [command-line](#) execution.

Imagine your VM is a **computer in the cloud** (Azure server).

SSH lets you **open its terminal from your own computer**.

Connect like this : ssh [ubuntu@20.204.xxx.xxx](#)

Port	Service	What it does
22	SSH	Remote login to server
80	HTTP	Normal websites
443	HTTPS	Secure websites
21	FTP	File transfer
25	SMTP	Email sending
3306	MySQL	Database access

A port is a numbered communication channel used by a computer to run different network services simultaneously.

Azure Student subscriptions restrict deployment to certain regions through policy rules. When the VM was deployed in a restricted region, Azure blocked the request. After choosing an allowed region, the VM deployment succeeded.

ssh hekhek@20.196.112.231

—>>

Update the system packages. Set up unattended upgrades to ensure the system always receives the latest security updates.

1-sudo apt update

sudo = superuser do (Linux has permission levels. Normal users cannot modify system software.)

apt = **Advanced Package Tool** : **package manager** used by Ubuntu/Debian systems.

A package manager is responsible for:

installing software,updating software,removing software,resolving dependencies

Example packages: nginx,python,git,docker

So when you install software in Ubuntu, it usually comes from **APT repositories**.

after **apt update**, your system **knows what updates exist**.This does **NOT update software**.It only **updates the package list**.

2-sudo apt upgrade -y

Now the system actually **installs updates**.APT compares: installed packages Vs latest packages in repository.Then it upgrades the outdated ones.

-y : yes to all prompts

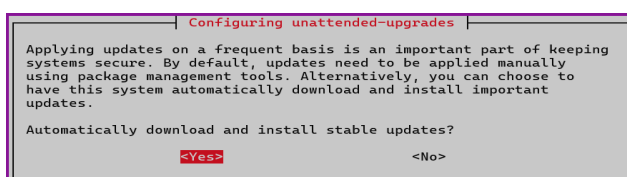
3-sudo apt install unattended-upgrades -y

Install : APT downloads and installs software from repositories.

Unattended-upgrades : It is a **background service** that automatically installs **security updates**.

Without it,You must manually run: apt update,apt upgrade & With unattended upgrades:The system checks for updates **automatically**.

4-sudo dpkg-reconfigure unattended-upgrades



dpkg = **Debian Package Manager** :It is the **low-level package system** used by APT.

Reconfigure ; command **runs the configuration wizard again**.

5-

```
hekhek@hekhek:~$ sudo systemctl status unattended-upgrades
● unattended-upgrades.service - Unattended Upgrades Shutdown
   Loaded: loaded (/lib/systemd/system/unattended-upgrades.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2026-03-15 12:07:30 UTC; 31min ago
     Docs: man:unattended-upgrade(8)
  Main PID: 787 (unattended-upgr)
    Tasks: 2 (limit: 1004)
   Memory: 8.3M
      CPU: 74ms
   CGroup: /system.slice/unattended-upgrades.service
           └─787 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade

Mar 15 12:07:30 hekhek systemd[1]: Started Unattended Upgrades Shutdown.
lines 1-12/12 (END)
```

At the bottom of your screen you can see: lines 1-12/12 (END)

That means the output is being shown in a program called **less** (a pager used to scroll long text).

While inside this viewer, the terminal **doesn't accept normal commands**. To return,press q(quit)

WHY LIKE THIS?

Commands like: `systemctl status` ,`man`,`git log` ,`journalctl` often produce **long outputs**.Instead of dumping everything directly, Linux opens a **pager (less)** so you can scroll.

A **terminal pager**, **paging program** or simply **pager** is a **computer program** used to view (but not modify) the contents of a **text file** moving down the file one line or one screen at a time. Some, but not all, pagers allow movement up a file.

A popular **cross-platform** terminal pager is **more**, which can move forwards and backwards in text files but cannot move backwards in **pipes**. **less** is a more advanced pager that allows movement forward and backward, and contains extra functions such as search.

systemctl is the **command-line interface to control systemd**.

- **systemd** → the **init system** in most modern Linux distributions (Ubuntu, Debian, Fedora, etc.).

- It is responsible for: Starting services during boot, Managing background processes, Handling system states, Managing daemons (services)

systemctl = tool to control systemd

Think of it like a **remote control for system services**.

Example services: **ssh** → SSH server, **nginx** → web server, **docker** → container engine, **cron** → scheduled tasks, **ufw** → firewall

systemctl <command> <service_name>

systemctl start nginx (systemctl → program, start → action, nginx → service)

Another example: systemctl status ssh (Check the status of the **SSH service**.)

22-03-26

Task2: SSH Configuration and MFA

OpenSSH is the actual software implementation of SSH.

OpenBSD is a security-focused, free software, Unix-like operating system based on the Berkeley Software Distribution (BSD).

Firewall is a network security system that monitors and controls incoming and outgoing network traffic based on configurable security rules. A firewall typically establishes a barrier between a trusted network and an untrusted network, such as the Internet or between several Virtual LANs. Firewalls can be categorized as network-based or host-based.

Host-based firewalls are deployed directly on the host itself to control network traffic or other computing resources. This can be a daemon or service as a part of the operating system or an agent application for protection.

Network-based firewalls are positioned between two or more networks, typically between the local area network (LAN) and wide area network (WAN), their basic function being to control the flow of data between connected networks.

PF (Packet Filter, also written pf) is a BSD licensed stateful packet filter, a central piece of software for firewalling. PF was developed for OpenBSD, but has been ported to many other operating systems.

-> `ssh user@example.com`

Here are some common options you can use with the `ssh` command:

- `-p` - Specify the port [By default, SSH uses port 22.]
- `-i` - Use a private key file [eg: `ssh -i /path/to/private_key user@example.com`]
- `-v` - Enable verbose mode [provides detailed information about the SSH connection process ; shows everything happening internally]
- `-C` - Enable compression [can speed up data transfer by reducing the amount of data sent over the network]
- `-X` - Enable X11 forwarding [enabling you to run graphical applications on the remote server and display them locally]
- `-o` - Specify options directly on the command line [allows you to specify SSH options directly on the command line. This is useful for overriding configuration settings. Eg ; `ssh -o StrictHostKeyChecking=no user@example.com`]

The **scp** command is used to securely copy files between hosts on a network.

Eg: **scp** file.txt user@example.com:/home/user/

All options except **-o** and **-l** as extra- Limit the bandwidth used by the copy

The **curl** command is used to transfer data from or to a server using various protocols like HTTP, HTTPS, FTP, and more. It's a versatile tool for downloading files, testing APIs, and more.

To retrieve a web page, use **curl url: curl**
<http://example.com/file.txt>

The **curl** command has options to change how it works:

- **-O** - Save the file with the same name as the remote file
- **-L** - Follow redirects [This is useful when accessing URLs that may redirect to another location.]
- **-I** - Fetch the HTTP headers only [This is useful for checking server response headers without downloading the entire content]

[user@localhost] \$	curl -I http://example.com
	# Output:
	# HTTP/1.1 200 OK
	# Date: Wed, 10 Apr 2025 10:00:00 GMT
	# Content-Type: text/html; charset=UTF-8
	# Connection: keep-alive

- **-d** - Send data with POST request

[user@localhost] \$	curl -d "fname=John" https://www.example.com/action_page.php
	Submitted Form Data
	Your input was received as:
	fname=John

- **-u** - Specify user and password for server authentication [useful for accessing protected resources.]

-> ssh -V [v - verbose mode, V- version check]

OpenSSH_8.9p1 Ubuntu-3ubuntu0.14, OpenSSL 3.0.2 15 Mar 2022

27-03-26

Setting up SSH (Secure Shell);

-> mkdir -p ~/.ssh (Create a **.ssh** folder in my home directory if it doesn't already exist.)

-p : mkdir only if it doesn't exist

-> chmod 700 ~/.ssh

chmod → changes file/directory permissions

700 → permission code:

- 7 (owner) = read + write + execute
- 0 (group) = no permissions
- 0 (others) = no permissions

NOTE : mkdir -m 700 mydir = mkdir mydir , chmod 700 mydir

-m : mode/permissions

chmod u+x file # add execute for user

chmod go-r file # remove read from group & others

chmod a=r file # everyone read-only

- touch filename { Creates a new empty file OR updates the timestamp of an existing file }

The **~/.ssh** directory stores SSH configuration files, keys, and known hosts.
The 700 permission restricts access to the owner only.

“ SSH reads configurations in this order:

- Command-line options (highest priority)
- User configuration file (**~/.ssh/config**)
- System-wide configuration file (**/etc/ssh/ssh_config**) {Options specified earlier override later ones.}

BEST PRACTICE

Always backup existing SSH configurations before making changes. Use `cp ~/.ssh/config ~/.ssh/config.backup` to create a safe copy.

/etc/ssh/ → Server-side (system-wide SSH)

This directory belongs to the SSH server (sshd).

👉 It controls how your VM behaves when someone connects to it

~/.ssh/ → User-side (your personal SSH)

This is your personal SSH directory

👉 It controls:

- how YOU connect to other systems
- who can connect to YOU

CREATE MAIN SSH CLIENT CONFIGURATION FILE

```
touch ~/.ssh/config  
chmod 600 ~/.ssh/config
```

(only r+w)

28-03-26

LOCKED OUT??

```
hekhkek@hehek:~/.ssh$ ls -ld ~/.ssh
ls -l ~/.ssh/authorized_keys
ls -ld ~
drwx----- 2 hekhkek hekhkek 4096 Mar 28 05:49 /home/hekhkek/.ssh
-rw-rw-r-- 1 hekhkek hekhkek 96 Mar 28 05:52 /home/hekhkek/.ssh/authorized_keys
drwxr-x--- 5 hekhkek hekhkek 4096 Mar 27 21:28 /home/hekhkek
```

Now in Serial Console

During configuration, SSH key-based authentication failed and the system kept prompting for a password. Eventually, access was lost due to:

- Deletion of the original **authorized_keys** file
- Misconfiguration of permissions
- Incorrect username usage during SSH login

authorized_keys had: -rw-rw-r--

SSH requires: -rw-----

Otherwise, SSH ignores the file.

NANO :

When working on the command line, quite often you will need to create or edit text files. Two of the most powerful and popular command-line editors are Vim and Emacs. Both have a steep learning curve that can be intimidating to new users. For those who need a simple editor, there is nano.

nano [OPTIONS] [FILE] ; general syntax

If you want to open a file with the cursor on a specific line and character, use the following syntax:

```
$ nano +line_number,character_number filename
```

For example, `nano +20 filename` opens the file at line 20, and `nano +20,5 filename` opens at line 20, column 5. If you omit the column number, the cursor will be positioned on the first character.

systemctl - Control the systemd system and service manager

`sudo systemctl restart ssh : restart ssh`

```
vivek@nixcraft:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 17.10
Release:        17.10
Codename:       artful
vivek@nixcraft:~$ sudo systemctl stop ssh
[sudo] password for vivek:
vivek@nixcraft:~$ sudo systemctl start ssh
vivek@nixcraft:~$ sudo systemctl reload ssh
vivek@nixcraft:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2018-05-28 02:02:28 IST; 10s ago
     Process: 12478 ExecReload=/bin/kill -HUP $MAINPID (code=exited, status=0/SUCCESS)
     Process: 12477 ExecReload=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
     Process: 12421 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
    Main PID: 12422 (sshd)
      Tasks: 1 (limit: 4915)
     Memory: 1.3M
        CPU: 26ms
    CGroup: /system.slice/ssh.service
            └─12422 /usr/sbin/sshd -D

May 28 02:02:28 nixcraft systemd[1]: Starting OpenBSD Secure Shell server...
May 28 02:02:28 nixcraft sshd[12422]: Server listening on 0.0.0.0 port 22.
May 28 02:02:28 nixcraft sshd[12422]: Server listening on :: port 22.
May 28 02:02:28 nixcraft systemd[1]: Started OpenBSD Secure Shell server.
May 28 02:02:35 nixcraft systemd[1]: Reloading OpenBSD Secure Shell server.
May 28 02:02:35 nixcraft sshd[12422]: Received SIGHUP; restarting.
May 28 02:02:35 nixcraft systemd[1]: Reloaded OpenBSD Secure Shell server.
May 28 02:02:35 nixcraft sshd[12422]: Server listening on 0.0.0.0 port 22.
May 28 02:02:35 nixcraft sshd[12422]: Server listening on :: port 22.
vivek@nixcraft:~$
```

© www.cyberciti.biz

```
#Privilege Separation is turned on for security
UsePrivilegeSeparation yes

# Lifetime and size of ephemeral version 1 server key
KeyRegenerationInterval 3600
ServerKeyBits 1024

# Logging
SyslogFacility AUTH
LogLevel INFO

# Authentication:
LoginGraceTime 120
PermitRootLogin without-password
StrictModes yes

RSAAuthentication yes
PubkeyAuthentication yes
#AuthorizedKeysFile      %h/.ssh/authorized_keys

# Don't read the user's ~/.rhosts and ~/.shosts files
IgnoreRhosts yes
# For this to work you will also need host keys in /etc/ssh_known_hosts
RhostsRSAAuthentication no
# similar for protocol version 2
HostbasedAuthentication no
# Uncomment if you don't trust ~/.ssh/known_hosts for RhostsRSAAuthentication
#IgnoreUserKnownHosts yes

# To enable empty passwords, change to yes (NOT RECOMMENDED)
PermitEmptyPasswords no

# Change to yes to enable challenge-response passwords (beware issues with
# some PAM modules and threads)
ChallengeResponseAuthentication no

# Change to no to disable tunnelled clear text passwords

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
(etc/ssh/sshd_config)
```

```
PermitRootLogin no
```

```
PasswordAuthentication no
```

```
PubkeyAuthentication yes
```

Also, removed RSA Authentication line

For next step (RESTRICTING IP)

What is an IP address?

IP address (or Internet Protocol address) is a numerical address of your device network connection. Basically, it's a set of rules that serves as a way of your device identification on the internet.

How IP addresses work?

You can compare Internet Protocol to postal services in the real-world. A postal code is an identifier for mail routing. And, in very simple words, IP address is a digital postal code for internet traffic routing. Every device, website and network server have an IP address.

How an IP address looks like?

There are 2 common standards for IP address: IPv4 (IP Version 4) and IPv6 (IP Version 6) that is aimed at replacing IPv4 in the future.

IPv4 address uses 32 binary bits and contains from 4 to 12 digits, 4 numbers separated by dots.

Examples: 8.8.4.4, 188.100.210.213

IPv6 address uses 128 binary bits and can contain up to 45 characters (8 groups of 4 hexadecimal digits separated by 6 colons). Groups with all zeros can be shortened for the simplicity of the representation.

Example: 2001:4860:4860::8888 (expanded – 2001:4860:4860:0000:0000:0000:8888)

Restricting SSH access to a specific IP address can significantly improve security by limiting connections to only trusted sources; however, it is not always practical in environments where the public IP is dynamic. In typical home or mobile networks, Internet Service Providers assign IP addresses dynamically using DHCP, meaning the IP can change when reconnecting to the network or switching networks (e.g., WiFi to hotspot). This would result in losing access to the server unless the allowed IP is continuously updated, which is inconvenient and error-prone. Therefore, in such scenarios, it is more practical to rely on strong authentication mechanisms like SSH key-based login and additional protections such as fail2ban, rather than strict IP-based restrictions.

The Dynamic Host Configuration Protocol (DHCP) is a **network management protocol** used on **Internet Protocol (IP)** networks for automatically assigning **IP addresses** and other communication parameters to devices connected to the network using a **client-server** architecture.

- IP restriction can be applied using CIDR notation (e.g., **106.76.0.0/16**) to allow a range of addresses instead of a single IP.

(first 16 bits are fixed, 106.76.0.0-106.76.255.255)

CIDR(Classless Inter Domain Routing) is a method of IP address allocation and routing that allows more efficient use of IP addresses. Unlike traditional class-based addressing, CIDR allocates IP addresses based on a network prefix rather than a fixed class (A, B, or C).

- **Public IPs are usually dynamic (DHCP), meaning they change when switching networks or over time.**
- **Using a small range may block your access if your IP changes, while a large range reduces security by allowing more users.**
- Restricting access by country (e.g., India) is not reliable with basic tools like UFW, as IPs are not strictly tied to geographic boundaries.

Ubuntu ships with a firewall configuration tool called **UFW (Uncomplicated Firewall)**. It is a user-friendly front-end for managing **iptables** firewall rules. Its main goal is to make managing a firewall easier or, as the name says, uncomplicated.

Therefore, IP restriction is often less practical, and SSH keys + fail2ban provide a more reliable and secure solution.

Real-world:

- **Companies restrict SSH to office IP / VPN only**
- **Massive reduction in attack surface**

Ip tables : The `iptables` command in Linux is a powerful tool that is used for managing the firewall rules and network traffic. It facilitates allowing the administrators to configure rules that help how packets are filtered, translated, or forwarded. On using this `iptables`, you can set up security policies to control incoming and outgoing traffic, define port forwarding, and implement network address translation (NAT). It's essential for securing servers and networks by selectively permitting or denying specific types of traffic based on defined rules and conditions. This flexibility makes `iptables` a fundamental component in Linux networking and security configurations.

<https://www.geeksforgeeks.org/linux-unix/iptables-command-in-linux-with-examples/>

Ports : Ports are logical communication endpoints used by transport layer protocols (TCP-Transmission Control Protocol/UDP-User Datagram Protocol) to identify specific services or applications on a device. Different types of ports are used in network to make a clean way to identify the right service or application, avoid conflicts, and manage connections efficiently. They are classified based on their functionality and usage in computer systems and networks.

Physical(USB,Ethernet,HDMI,VGA,PS/2,etc),

Logical(Dynamic,Internal,External,Well-known,Registered,serial/parallel,etc),

Virtual(software-defined "tunnels" used in **Virtual Machines (VMs)** or **Cloud Computing** to allow different virtual OSs running on the same hardware to talk to each other without getting their signals crossed)

There are **65,535 ports** on a machine: 0 – 65535

But: **Most ports = closed** ; Only some are: **open (allowed by firewall) , listening (service running)**

```
nekhek@hehek:~$ ss -tuln
```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
udp	UNCONN	0	0	127.0.0.53%lo:53	0.0.0.0:*	
udp	UNCONN	0	0	10.0.0.4%eth0:68	0.0.0.0:*	
udp	UNCONN	0	0	127.0.0.1:323	0.0.0.0:*	
udp	UNCONN	0	0	:::1:323	:::]:*	
tcp	LISTEN	0	4096	127.0.0.53%lo:53	0.0.0.0:*	
tcp	LISTEN	0	128	0.0.0.0:22	0.0.0.0:*	
tcp	LISTEN	0	128	:::]:22	:::]:*	

The Domain Name System (DNS) is the phonebook of the Internet. Humans access information online through **domain names**, like nytimes.com or espn.com. Web browsers interact through **Internet Protocol (IP)** addresses. DNS translates domain names to **IP addresses** so browsers can load Internet resources.

ss (socket statistics) is a command-line utility to investigate sockets in the Linux system. It displays similar information to **netstat** command. It can show more TCP and state information.

A network socket is a software structure within a **network node** of a **computer network** that serves as an endpoint for sending and receiving data across the network. The structure and properties of a socket are defined by an **application programming interface** (API) for the networking architecture. Sockets are created only during the lifetime of a **process** of an application running in the node.

Because of the **standardization** of the **TCP/IP** protocols in the development of the **Internet**, the term **network socket** is most commonly used in the context of the Internet protocol suite, and is therefore often also referred to as Internet socket. In this context, a socket is externally identified to other hosts by its socket address, which is the triad of **transport protocol**, **IP address**, and **port number**.

bash

Copy

```
$ ss [option]
```

Some of the important options in `ss` command are:

- **-a**: To display all sockets
- **-l**: To display only listening sockets
- **-t**: To display only TCP sockets
- **-u**: To display only UDP sockets
- **-x**: To display only UNIX domain sockets
- **-m**: To display socket memory usage
- **-s**: To display summary statistics

Setting up UFW :

```
hekhhek@hehek:~$ sudo ufw status
Status: inactive
hekhhek@hehek:~$ sudo ufw allow 22
Rules updated
Rules updated (v6)
hekhhek@hehek:~$ sudo ufw allow 80
Rules updated
Rules updated (v6)
hekhhek@hehek:~$ sudo ufw allow 443
Rules updated
Rules updated (v6)
hekhhek@hehek:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)?
y
Firewall is active and enabled on system startup
hekhhek@hehek:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22 ALLOW IN Anywhere
80 ALLOW IN Anywhere
443 ALLOW IN Anywhere
22 (v6) ALLOW IN Anywhere (v6)
80 (v6) ALLOW IN Anywhere (v6)
443 (v6) ALLOW IN Anywhere (v6)
```

Our server is listening to both ipv4 and v6 that's why UFW created rules for both.


```

hekhek@hekhek:~$ sudo ufw allow 2222
Rule added
Rule added (v6)
hekhek@hekhek:~$ sudo nano /etc/ssh/sshd_config
hekhek@hekhek:~$
hekhek@hekhek:~$ sudo systemctl restart ssh
hekhek@hekhek:~$ ssh -tulin
unknown option -- u
usage: ssh [-46AaCfGgKkMMNnqsTtVvXxYy] [-B bind_interface]
          [-b bind_address] [-c cipher_spec] [-D [bind_address:]port]
          [-E log_file] [-e escape_char] [-F configfile] [-I pkcs11]
          [-i identity_file] [-J [user@]host[:port]] [-L address]
          [-l login_name] [-m mac_spec] [-O ctl_cmd] [-o option] [-p port]
          [-Q query_option] [-R address] [-S ctl_path] [-W host:port]
          [-w local_tun[:remote_tun]] destination [command [argument ...]]
hekhek@hekhek:~$ ss -tulin
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
udp UNCONN 0 0 127.0.0.53%lo:53 0.0.0.0:*
udp UNCONN 0 0 10.0.0.4%eth0:68 0.0.0.0:*
udp UNCONN 0 0 127.0.0.1:323 0.0.0.0:*
udp UNCONN 0 0 [::1]:323 [::]:*
tcp LISTEN 0 4096 127.0.0.53%lo:53 0.0.0.0:*
tcp LISTEN 0 128 0.0.0.0:2222 0.0.0.0:*
tcp LISTEN 0 128 [::]:2222 [::]:*
hekhek@hekhek:~$ ss -tulin | grep 2222
tcp LISTEN 0 128 0.0.0.0:2222 0.0.0.0:*
tcp LISTEN 0 128 [::]:2222 [::]:*
hekhek@hekhek:~$ sudo ufw status
Status: active

To Action From
--
22 ALLOW Anywhere
80 ALLOW Anywhere
443 ALLOW Anywhere
2222 ALLOW Anywhere
22 (v6) ALLOW Anywhere (v6)
80 (v6) ALLOW Anywhere (v6)
443 (v6) ALLOW Anywhere (v6)

```

I tried to change the port from 22 to 2222 in sshd_config ; before, added ufw allow 2222 but even after systemctl restart ssh, it wasn't opening using 2222(connection timedout) and 22(closed) as well.Anyway changed back to 22-working fine;

- SSH was successfully running on port 2222 (verified using `ss -tulin`)
- However, connection to port 2222 failed with timeout
- Root cause:
 - Azure has a Network Security Group (NSG) acting as a cloud-level firewall
 - NSG was still allowing port 22 only, blocking 2222
- Result:
 - Port 22 stopped working (since SSH moved away)
 - Port 2222 unreachable → temporary lockout risk
- Resolution:
 - Either:
 - Add port 2222 in Azure NSG, OR
 - Revert SSH back to port 22 (safe option)
- Key lesson:
 - Two firewalls exist → both must allow traffic:
 - VM firewall (UFW)
 - Cloud firewall (Azure NSG)

ssh → connect to server : -p 2222 → use port 2222 instead of default 220

- | grep 2222 : Filters only lines containing 2222

Eg : Find SSH port : ss -tuln | grep ssh

grep = search tool

grep "text" ; Filters lines containing that text

Inbound port rules (5)							
300	SSH		22	TCP	Any	Any	Allow
310	allo-ssh-2222		2222	TCP	Any	Any	Allow

After adding inbound port rules-IT WORKED!!

Earlier:

- ❌ Azure blocked 2222 → timeout

Now:

- ✅ Azure allows 2222
- ✅ UFW allows 2222
- ✅ SSH listens on 2222

```
hehkek@hehkek:~$ sudo ufw status numbered
Status: active

To Action From
--
[ 1] 22 ALLOW IN Anywhere
[ 2] 80 ALLOW IN Anywhere
[ 3] 443 ALLOW IN Anywhere
[ 4] 2222 ALLOW IN Anywhere
[ 5] 22 (v6) ALLOW IN Anywhere (v6)
[ 6] 80 (v6) ALLOW IN Anywhere (v6)
[ 7] 443 (v6) ALLOW IN Anywhere (v6)
[ 8] 2222 (v6) ALLOW IN Anywhere (v6)

hehkek@hehkek:~$ sudo ufw delete allow 22/tcp
Could not delete non-existent rule
Could not delete non-existent rule (v6)
hehkek@hehkek:~$ sudo ufw delete 1
Deleting:
allow 22
Proceed with operation (y/n)? y
Rule deleted
hehkek@hehkek:~$ sudo ufw status numbered
Status: active

To Action From
--
[ 1] 80 ALLOW IN Anywhere
[ 2] 443 ALLOW IN Anywhere
[ 3] 2222 ALLOW IN Anywhere
[ 4] 22 (v6) ALLOW IN Anywhere (v6)
[ 5] 80 (v6) ALLOW IN Anywhere (v6)
[ 6] 443 (v6) ALLOW IN Anywhere (v6)
[ 7] 2222 (v6) ALLOW IN Anywhere (v6)
```

Now deleted 22

\

sudo ufw default deny incoming [Block ALL incoming traffic unless I explicitly allow it.]

default deny policy” (zero trust)

sudo ufw default allow outgoing [My server can send traffic anywhere freely]

The default deny incoming policy ensures that only explicitly permitted traffic is allowed into the system, improving security by minimizing attack surfaces. The default allow outgoing policy ensures normal system functionality like updates and external communication.

If port 80 is not allowed:

```
Client → tries HTTP request → port 80
↓
Hits UFW
↓
No matching ALLOW rule
↓
Default policy = DENY
↓
Connection dropped
```

👉 So the browser will show:

- ❌ "Connection timed out" OR
- ❌ "Site can't be reached"

Now enabling logging

->

```
hekkhek@hehek:~$ sudo ufw logging on
Logging enabled
hekkhek@hehek:~$ sudo tail -f /var/log/ufw.log
tail: cannot open '/var/log/ufw.log' for reading: No such file or directory
tail: no files remaining
hekkhek@hehek:~$ sudo tail -f /var/log/ufw.log
tail: cannot open '/var/log/ufw.log' for reading: No such file or directory
tail: no files remaining
hekkhek@hehek:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
80 ALLOW IN Anywhere
443 ALLOW IN Anywhere
2222 ALLOW IN Anywhere
80 (v6) ALLOW IN Anywhere (v6)
443 (v6) ALLOW IN Anywhere (v6)
2222 (v6) ALLOW IN Anywhere (v6)
```

- 👉 UFW doesn't always create a separate file (ufw.log)
- 👉 Instead, it logs into:

```
👉 Bash
/var/log/syslog
```

```
hekkhek@hehek:~$ sudo tail -f /var/log/ufw.log
tail: cannot open '/var/log/ufw.log' for reading: No such file or directory
tail: no files remaining
hekkhek@hehek:~$ cd ..
hekkhek@hehek:/home$ cd ..
hekkhek@hehek:/home$ ls
bin dev home lib32 libx32 media opt root sbin srv tmp var
boot etc lib lib64 lost+found mnt proc run snap sys usr
hekkhek@hehek:/home$ cd /var
hekkhek@hehek:/var$ ls
backups crash local log opt snap tmp
cache lib lock mail run spool
hekkhek@hehek:/var$ cd log
hekkhek@hehek:/var/log$ ls
alternatives.log dmesg lastlog
apt dmesg.0 private
auth.log dmesg.1.gz syslog
auth.log.1 dmesg.2.gz syslog.1
azure dpkg.log sysstat
btmpt fail2ban.log ubuntu-advantage-apt-hook.log
chrony journal unattended-upgrades
cloud-init-output.log kern.log waagent.log
cloud-init.log kern.log.1 wtmp
dist-upgrade landscape
hekkhek@hehek:/var/log$ cd ..
hekkhek@hehek:/var$ cd
hekkhek@hehek:~$ sudo tail -f /var/log/syslog
Mar 28 12:15:33 hehek kernel: [23519.231531] [UFW BLOCK] IN=eth0 OUT= MAC=00:22:48:05:43:fc:12:34:56:78:9a:bc:08:00 SRC=
=0x00 PREC=0x00 TTL=101 ID=15094 DF PROTO=TCP SPT=9249 DPT=22 WINDOW=65535 RES=0x00 SYN URGP=0
Mar 28 12:15:35 hehek kernel: [23521.031306] [UFW BLOCK] IN=eth0 OUT= MAC=00:22:48:05:43:fc:12:34:56:78:9a:bc:08:00 SRC=
=0x00 PREC=0x00 TTL=101 ID=15095 DF PROTO=TCP SPT=9249 DPT=22 WINDOW=65535 RES=0x00 SYN URGP=0
Mar 28 12:15:39 hehek kernel: [23525.258010] [UFW BLOCK] IN=eth0 OUT= MAC=00:22:48:05:43:fc:12:34:56:78:9a:bc:08:00 SRC=
```

BLOCK = denied

SRC = attacker/client IP

DPT = destination port

UFW logs can be stored either in `/var/log/ufw.log` or `/var/log/syslog` depending on system configuration. In this setup, logs are being recorded in syslog, which still fulfills auditing requirements.\

If you want a dedicated file:

```
sudo touch /var/log/ufw.log  
sudo chmod 640 /var/log/ufw.log
```

Fail2Ban is an [intrusion prevention](#) software framework. Written in the [Python](#) programming language, it is designed to prevent [brute-force attacks](#). It is able to run on [POSIX](#) systems that have an interface to a packet-control system or firewall installed locally, such as [iptables](#) or [TCP Wrapper](#)

The main functions of intrusion prevention systems(IPS/IDPS) are to identify malicious activity, log information about this activity, report it and attempt to block or stop it.

In [cryptography](#), a **brute-force attack** or **exhaustive key search** is a [cryptanalytic attack](#) that consists of an attacker submitting many possible [keys](#) or [passwords](#) with the hope of eventually guessing correctly. This strategy can theoretically be used to break any form of encryption that is not [information-theoretically secure](#).

An encryption protocol with information-theoretic security is impossible to break even with infinite computational power. Protocols proven to be information-theoretically secure are resistant to future developments in computing (Vulnerable only to physical theft of the key.) eg : OTP(One-Time-Pad), QKD(Quantum-Key-Distribution)

TCP Wrappers (also known as **tcp_wrappers**) is a host-based networking [ACL](#) system, used to [filter](#) network access to [Internet Protocol](#) servers on ([Unix-like](#)) [operating systems](#) such as [Linux](#) or [BSD](#). It allows host or [subnetwork IP addresses](#), [names](#) and/or [ident](#) query replies, to be used as tokens on which to filter for [access control](#) purposes.

Attacker tries password many times → FAIL ; fail2ban → detects → blocks IP automatically;

[Uses logs : Fail2ban operates by monitoring [log files](#) (e.g. `/var/log/auth.log`, `/var/log/apache/access.log`, etc.) for selected entries and running scripts based on them.]

```

hekhek@hekhek:~$ sudo systemctl start fail2ban
hekhek@hekhek:~$ sudo systemctl enable fail2ban
Synchronizing state of fail2ban.service with SysV service script with /lib/s
ystemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable fail2ban
Created symlink /etc/systemd/system/multi-user.target.wants/fail2ban.service
→ /lib/systemd/system/fail2ban.service.
hekhek@hekhek:~$ sudo systemctl status fail2ban
● fail2ban.service - Fail2Ban Service
   Loaded: loaded (/lib/systemd/system/fail2ban.service; enabled; vendor >
   Active: active (running) since Sat 2026-03-28 08:29:44 UTC; 22s ago
     Docs: man:fail2ban(1)
    Main PID: 13877 (fail2ban-server)
       Tasks: 5 (limit: 1009)
      Memory: 14.1M
         CPU: 137ms
    CGroup: /system.slice/fail2ban.service
            └─13877 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

Mar 28 08:29:44 hekhek systemd[1]: Started Fail2Ban Service.
Mar 28 08:29:45 hekhek fail2ban-server[13877]: Server ready

hekhek@hekhek:~$ sudo fail2ban-client status
Status
|- Number of jail:      1
`- Jail list:  sshd
hekhek@hekhek:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 1
|  |- Total failed:    1
|  `-- File list:      /var/log/auth.log
`- Actions
   |- Currently banned: 0
   |- Total banned:    0
   `-- Banned IP list:

```

When will fail2ban BAN someone?

Default logic: If failed attempts > maxretry (usually 5) , within findtime (usually 10 mins) → BAN

```

hekhek@hekhek:~$ sudo tail -f /var/log/auth.log
Mar 28 08:34:50 hekhek sshd[14006]: rexec line 18: Deprecated option UsePrivilegeSeparation
Mar 28 08:34:50 hekhek sshd[14006]: rexec line 21: Deprecated option KeyRegenerationInterval
Mar 28 08:34:50 hekhek sshd[14006]: rexec line 22: Deprecated option ServerKeyBits
Mar 28 08:34:50 hekhek sshd[14006]: rexec line 39: Deprecated option RhostsRSAAuthentication
Mar 28 08:34:50 hekhek sshd[14006]: error: Unable to load host key: /etc/ssh/ssh_host_dsa_key
Mar 28 08:34:51 hekhek sshd[14006]: reprocess config line 39: Deprecated option RhostsRSAAuthentication
Mar 28 08:34:51 hekhek sshd[14006]: Connection reset by authenticating user root 2.57.122.195 port 53536 [preauth]
Mar 28 08:35:01 hekhek CRON[14008]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
Mar 28 08:35:01 hekhek CRON[14008]: pam_unix(cron:session): session closed for user root
Mar 28 08:36:18 hekhek sudo: hekhek : TTY=pts/0 ; PWD=/home/hekhek ; USER=root ; COMMAND=/usr/bin/tail -f /var/log/auth.log
Mar 28 08:36:18 hekhek sudo: pam_unix(sudo:session): session opened for user root(uid=0) by hekhek(uid=1000)
Mar 28 08:36:29 hekhek sshd[14014]: rexec line 18: Deprecated option UsePrivilegeSeparation
Mar 28 08:36:29 hekhek sshd[14014]: rexec line 21: Deprecated option KeyRegenerationInterval
Mar 28 08:36:29 hekhek sshd[14014]: rexec line 22: Deprecated option ServerKeyBits
Mar 28 08:36:29 hekhek sshd[14014]: rexec line 39: Deprecated option RhostsRSAAuthentication
Mar 28 08:36:29 hekhek sshd[14014]: error: Unable to load host key: /etc/ssh/ssh_host_dsa_key
Mar 28 08:36:31 hekhek sshd[14014]: reprocess config line 39: Deprecated option RhostsRSAAuthentication
Mar 28 08:36:31 hekhek sshd[14014]: Connection reset by authenticating user root 2.57.122.192 port 57772 [preauth]

```

Tail Command in Linux is used to display the last part of a file, showing recent content such as logs or updates.

- By default, it shows the last 10 lines of a file.
- Commonly used for monitoring log files and debugging.
- You can customize the number of lines displayed using the -n option.
- Useful for viewing the most recent entries without opening the entire file.

- The `-f` option in the `tail` command is used to continuously monitor a file in real-time.
- It displays the last 10 lines of a file and automatically updates as new lines are added. (Doesn't quit unless `ctrl+c`)

The Digital Signature Algorithm (DSA) is a Federal Information Processing Standard that governs digital signatures. It is used to ensure the validity and integrity of a messages, software, or digital document. The National Institute of Standards and Technology (NIST) suggested DSA in August 1991 for use in the Digital Signature Standard (DSS). Digital Signature is a verification method. It does not provide confidential communication. If you want to achieve confidentiality, both the message and the signature must be encrypted using either a secret key or a public key cryptosystem. This additional layer of security can be incorporated into a basic digital signature scheme.

These two are standard Approaches to implement the Digital Signature:

- Rivest-Shamir-Adleman (RSA)
- Digital Signature Algorithm (DSA)

In our logs : Unable to load host key: /etc/ssh/ssh_host_dsa_key

Meaning: SSH tried to use DSA key -It doesn't exist {DSA is deprecated anyway & Modern SSH uses: ED25519 , RSA }

How does SSH key-based authentication work, and can another person or device access the server by copying or using these keys?

SSH authentication uses a public-private key pair. The public key is stored on the server (**authorized_keys**), while the private key remains securely on your laptop. When you attempt to log in, the server sends a challenge, which your laptop signs using the private key. The server then verifies this using the stored public key—if it matches, access is granted. **Another device can only access the server if it has your private key.** The public key alone is useless for login. Therefore, the security depends entirely on keeping your private key safe and never sharing it.

`sudo fail2ban-client set sshd unbanip <IP>` [To unban manually]

Each jail is **independent** ; SSH jail → separate rules , Nginx jail → separate rules

Jail	Protects
sshd	SSH login
nginx-http-auth	web login
apache	web server
vsftpd	FTP
postfix	mail

Service (SSH) → needs protection ; fail2ban → creates a jail for it

Each jail = filter (what to watch) + rules (times,limit) + action (ban)

MFA-Multi-Factor-Authentication :

Login = SSH key + OTP (one-time password)

-> google-authenticator

Do u want authentication tokens to be time-based? - y

```
Do you want me to update your "/home/hekhek/.google_authenticator" file? (y/n) y

Do you want to disallow multiple uses of the same authentication
token? This restricts you to one login about every 30s, but it increases
your chances to notice or even prevent man-in-the-middle attacks (y/n) y

By default, a new token is generated every 30 seconds by the mobile app.
In order to compensate for possible time-skew between the client and the server,
we allow an extra token before and after the current time. This allows for a
time skew of up to 30 seconds between authentication server and client. If you
experience problems with poor time synchronization, you can increase the window
from its default size of 3 permitted codes (one previous code, the current
code, the next code) to 17 permitted codes (the 8 previous codes, the current
code, and the 8 next codes). This will permit for a time skew of up to 4 minutes
between client and server.
Do you want to do so? (y/n) n

If the computer that you are logging into isn't hardened against brute-force
login attempts, you can enable rate-limiting for the authentication module.
By default, this limits attackers to no more than 3 login attempts every 30s.
Do you want to enable rate-limiting? (y/n) y
```

1. Update the `.google_authenticator` file? (y) **What it does:** This saves your secret key, emergency scratch codes, and the settings you just chose into a hidden file in your home directory. **Why it matters:** Without this, the system won't "remember" that you set up 2FA, and you won't be able to log in using your app.
2. Disallow multiple uses of the same token? (y) **What it does:** This makes each 6-digit code "single-use." Even if the 30-second timer hasn't run out, once you use a code to log in, that specific code becomes invalid. **The Security Win:** It prevents **Man-in-the-Middle (MITM)** attacks. If a hacker intercepts your code as you type it, they can't use it again immediately to start their own session.
3. Increase the Time-Skew window? (n) **What it does:** By default, the server accepts the *current* code, the *previous* one, and the *next* one (a 90-second window) to account for your phone's clock being slightly off from the server's clock. You chose "n", keeping this tight window. **The Trade-off:** * **Pros:** It's more secure. A smaller window means a smaller "time of opportunity" for a stolen code to work. **Cons:** If your phone's time drifts by even a minute, your codes will fail. *Tip:* If you ever find yourself locked out, make sure your phone is set to "Set time automatically."
4. Enable rate-limiting? (y) **What it does:** It limits a user to 3 login attempts every 30 seconds. **Why it matters:** It stops "brute-force" attacks. Even though a 6-digit code has 1 million combinations, a fast script could guess it eventually. Rate-limiting makes that impossible.

Multi-factor authentication (MFA), also known as two-factor authentication (2FA), is an [electronic authentication](#) method in which a user is granted access to a [website](#) or [application](#) only after successfully presenting two or more distinct types of evidence (or [factors](#)) to an [authentication](#) mechanism.

Linux Pluggable Authentication Modules (PAM) is a suite of libraries that allow a Linux [system administrator](#) to configure methods to [authenticate](#) users. It provides a flexible and centralized way to switch authentication methods for secured applications by using configuration files instead of changing application code.

PAM = a framework that decides HOW authentication happens in Linux

SSH → "User wants to login"

PAM → "How should I verify this user?"

1. SSH receives connection
2. SSH checks config (sshd_config)
3. SSH hands control to PAM
4. PAM decides authentication methods
5. Access granted/denied

A **pluggable authentication module (PAM)** is a mechanism to integrate multiple low-level [authentication](#) schemes into a high-level [application programming interface](#) (API). PAM allows programs that rely on authentication to be written independently of the underlying authentication scheme

A pam module = A plug-in that adds authentication logic

pam_unix.so	password auth
pam_google_authenticator.so	OTP
pam_limits.so	resource limits

(Module,Purpose)

Google Authenticator uses a protocol called **TOTP** (Time-based One-Time Password). It is a brilliant bit of math that allows two devices (the server and your phone) to agree on a 6-digit code without ever actually talking to each other at the moment of login.

1. The Shared Secret (The Setup) : When you first set up MFA and scan that QR code, your phone and the server exchange a **Shared Secret Key**. This key is a long string of random letters and numbers. It is stored securely on your phone and on Google's (or your own) server. **Crucial:** This is the *only* time the two devices need to communicate to set up the "logic" of your codes.

2. The Inputs: Time + Secret : To generate a code, the Authenticator app uses two pieces of information: **The Shared Secret:** The unique key from Step 1. **The Current Time:** Specifically, the number of 30-second "slices" that have passed since January 1, 1970 (Unix Epoch).

3. The "Blender" (HMAC-SHA1) The app puts the **Secret** and the **Current Time** into a mathematical "blender" called an **HMAC-SHA1** hashing algorithm. A hash is a one-way function. You can easily turn the "Secret + Time" into a complex string of data, but you can't reverse it to find the Secret. Because both your phone and the server have the *exact same* secret and the *exact same* time, they both get the *exact same* complex result from the blender.

4. Truncation (The 6-Digit Code) : The result of the hash is a long, messy string of data. Since humans hate typing 40-character hex codes, the algorithm performs "Dynamic Truncation." It grabs a specific section of that hash and converts it into a **6-digit number**. This is what you see on your screen.

5. Validation : When you type "123 456" into your computer: The server runs the same math at that exact second. If the server's 6-digit result matches yours, it knows you have the physical phone that holds the secret key. Access is granted.

Why is it so secure?

Offline: Your phone doesn't need Internet or a SIM card to generate codes. It only needs a working clock.

Information-Theoretic "Hint": As we discussed earlier, while not strictly ITS (because it relies on the SHA1 hash), it is practically unbreakable because the code expires every 30 seconds. Even if someone steals your code, it's useless a minute later.

In [cryptography](#), an HMAC (sometimes expanded as either keyed-hash message authentication code or hash-based message authentication code) is a specific type of [message authentication code](#) (MAC) involving a

cryptographic hash function and a secret cryptographic key. As with any MAC, it may be used to simultaneously verify both the data integrity and authenticity of a message. An HMAC is a type of keyed hash function that can also be used in a key derivation scheme or a key stretching scheme. **SHA-1 (Secure Hash Algorithm 1)** is a hash function which takes an input and produces a 160-bit (20-byte) hash value known as a message digest – typically rendered as 40 hexadecimal digits. **SHA-1** is indeed considered "broken" for certain uses. In 2017, Google researchers famously announced **SHAttered**, the first real-world collision attack where they produced two different PDF files with the same SHA-1 hash. To "break" Google Authenticator, a well-funded opponent doesn't just need a collision; they need a **Pre-image. SHA-1 is still extremely strong against Pre-image attacks.** Even with massive computing power, reversing the hash to find your secret key is computationally infeasible.

STEPS -

>sudo apt update

->sudo apt install libpam-google-authenticator -y

->google-authenticator

->after yyyny QR code Secret key Emergency codes

->sudo nano /etc/pam.d/sshd (enable pam)

Edit : auth required pam_google_authenticator.so this line at top #@include common_auth-comment

-.sudo nano /etc/ssh/sshd_config

ChallengeResponseAuthentication yes ; UsePAM yes ; AuthenticationMethods
publickey,keyboard-interactive (Add this line)

Then restart (sudo systemctl restart ssh)

CURRENT SSHD

#Default sshd_config

Package generated configuration file
See the sshd_config(5) manpage for details

What ports, IPs and protocols we listen for

Port 22

Use these options to restrict which interfaces/protocols sshd will bind to

#ListenAddress ::

#ListenAddress 0.0.0.0

Protocol 2 *[forces use of SSH Protocol 2, which is significantly more secure than the obsolete Protocol 1]*

HostKeys for protocol version 2

HostKey /etc/ssh/ssh_host_rsa_key

HostKey /etc/ssh/ssh_host_dsa_key

HostKey /etc/ssh/ssh_host_ecdsa_key

HostKey /etc/ssh/ssh_host_ed25519_key

[These point to the private keys the server uses to identify itself to clients. This prevents "impersonation" attacks where a fake server tries to steal your credentials.]

#Privilege Separation is turned on for security

UsePrivilegeSeparation yes

-A security feature that handles the initial connection in a "sandboxed" process with no privileges. If an attacker exploits a bug during login, they won't have root access.

Lifetime and size of ephemeral version 1 server key

KeyRegenerationInterval 3600

ServerKeyBits 1024

Logging

SyslogFacility AUTH **[/var/log/auth.log- tells system to log events there]**

LogLevel INFO **[sets verbosity of logs]**

Authentication:

LoginGraceTime 120

PermitRootLogin no **[No root login]**

StrictModes yes **{Should not be too [open](#). If world readable;refuses}**

PubkeyAuthentication yes **[allowas ssh keys-most secure]**

#AuthorizedKeysFile %h/.ssh/authorized_keys

AuthenticationMethods publickey,keyboard-interactive

[This is a "Multi-Factor" requirement. It tells the server: "To enter, the user MUST provide a public key AND complete a keyboard-interactive challenge (your Google Authenticator code).]

Don't read the user's ~/.rhosts and ~/.shosts files

IgnoreRhosts yes

For this to work you will also need host keys in /etc/ssh_known_hosts

RhostsRSAAuthentication no

similar for protocol version 2

HostbasedAuthentication no

Uncomment if you don't trust ~/.ssh/known_hosts for RhostsRSAAuthentication

#IgnoreUserKnownHosts yes

To enable empty passwords, change to yes (NOT RECOMMENDED)

PermitEmptyPasswords no

Change to yes to enable challenge-response passwords (beware issues with

some PAM modules and threads)

ChallengeResponseAuthentication yes

[This enables the "keyboard-interactive" part. Without this, your Google Authenticator setup won't work.]

Change to no to disable tunnelled clear text passwords
PasswordAuthentication no

Kerberos options
#KerberosAuthentication no
#KerberosGetAFSToken no
#KerberosOrLocalPasswd yes
#KerberosTicketCleanup yes

GSSAPI options
#GSSAPIAuthentication no
#GSSAPICleanupCredentials yes

X11Forwarding yes

[Allows you to run graphical Linux apps on the server and have the window pop up on your local computer.]

X11DisplayOffset 10

PrintMotd no

PrintLastLog yes

TCPKeepAlive yes

#UseLogin no

#MaxStartups 10:30:60

#Banner /etc/issue.net

Allow client to pass locale environment variables

AcceptEnv LANG LC_*

Subsystem sftp /usr/lib/openssh/sftp-server

Set this to 'yes' to enable PAM authentication, account processing,
and session processing. If this is enabled, PAM authentication will
be allowed through the ChallengeResponseAuthentication and
PasswordAuthentication. Depending on your PAM configuration,
PAM authentication via ChallengeResponseAuthentication may bypass
the setting of "PermitRootLogin without-password".

If you just want the PAM account and session checks to run without
PAM authentication, then enable this but set PasswordAuthentication
and ChallengeResponseAuthentication to 'no'.

UsePAM yes

CLOUD_IMG: This file was created/modified by the Cloud Image build process

ClientAliveInterval 120 explain each lines in details

IP TABLES

`iptables` is the actual firewall system in Linux.

- Works at kernel level
- Uses rules to filter packets

`iptables` is a **user-space** utility program that allows a **system administrator** to configure the **IP packet filter rules** of the **Linux kernel firewall**, implemented as different **Netfilter** modules.

Netfilter is a **framework** provided by the **Linux kernel** that allows various **networking**-related operations to be implemented in the form of customized handlers. Netfilter offers various functions and operations for **packet filtering**, **network address translation**, and **port translation**, which provide the functionality required for directing packets through a network and **prohibiting** packets from reaching sensitive locations within a network.

The filters are organized in a set of tables, which contain chains of rules for how to treat network traffic packets.

Different kernel modules and programs are currently used for different protocols; *iptables* applies to IPv4, *ip6tables* to IPv6, *arptables* to **ARP**, and *ebtables* to **Ethernet frames**.

`iptables` allows the **system administrator** to define *tables* containing *chains* of *rules* for the treatment of packets. Each table is associated with a **different kind of packet processing**. Packets are processed by sequentially traversing the rules in chains. A rule in a chain can cause a goto or jump to another chain, and this can be repeated to whatever level of nesting is desired. (A jump is like a “call”, i.e. the point that was jumped from is remembered.) Every network packet arriving at or leaving from the computer traverses at least one chain.

- The origin of the packet determines which chain it traverses initially. There are five *predefined chains* (mapping to the five available Netfilter hooks), though a table may not have all chains.
- Predefined chains have a *policy*, for example DROP, which is applied to the packet if it reaches the end of the chain.
- The system administrator can create as many other chains as desired. These chains have no policy; if a packet reaches the end of the chain it is returned to the chain which called it.
- A chain may be empty.
- **PREROUTING**: Packets will enter this chain before a routing decision is made.

- **INPUT:** Packet is going to be locally delivered. It does not have anything to do with processes having an opened socket; local delivery is controlled by the "local-delivery" routing table: `ip route show table local`.
- **FORWARD:** All packets that have been routed and were not for local delivery will traverse this chain.
- **OUTPUT:** Packets sent from the machine itself will be visiting this chain.
- **POSTROUTING:** Routing decision has been made. Packets enter this chain just before handing them off to the hardware.

A chain does not exist by itself; it belongs to a *table*. There are three tables: **nat**, **filter**, and **mangle**. Unless preceded by the option `-t`, an `iptables` command concerns the *filter* table by default. For example, the command `iptables -L -v -n`, which shows some chains and their rules, is equivalent to `iptables -t filter -L -v -n`. To show chains of table *nat*, use the command `iptables -t nat -L -v -n`.

Each rule in a chain contains the specification of which packets it matches. It may also contain a *target* (used for extensions) or *verdict* (one of the built-in decisions).

As a packet traverses a chain, each rule in turn is examined. If a rule does not match the packet, the packet is passed to the next rule.

If a rule does match the packet, the rule takes the action indicated by the target/verdict, which may result in the packet being allowed to continue along the chain or may not. Matches make up the large part of rulesets, as they contain the conditions packets are tested for.

These can happen for about any layer in the **OSI** (Open-Systems-Interconnection : the components of a communication system are distinguished in seven **abstraction layers**: Physical, Data Link, Network, Transport, Session, Presentation, and Application.) model, as with e.g. the `--mac-source` and `-p tcp --dport` parameters, and there are also protocol-independent matches, such as `-m time`.

The packet continues to traverse the chain until either

1. a rule matches the packet and decides the ultimate fate of the packet, for example by calling one of the `ACCEPT` or `DROP`, or a module returning such an ultimate fate; or
2. a rule calls the `RETURN` verdict, in which case processing returns to the calling chain; or

- the end of the chain is reached; traversal either continues in the parent chain (as if RETURN was used), or the base chain policy, which is an ultimate fate, is used.

Targets also return a verdict like ACCEPT (NAT modules will do this) or DROP (e.g. the REJECT module), but may also imply CONTINUE (e.g. the LOG module; CONTINUE is an internal name) to continue with the next rule as if no target/verdict was specified at all.

```
—iptables -A INPUT -p tcp --dport 2222 -j ACCEPT
```

[Append rule to INPUT chain ,If TCP packet on port 2222 → allow]

```
ufw default deny incoming = iptables -P INPUT DROP
```

```
sudo ufw allow 80/tcp = iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

Ufw translates your commands into iptables rules;

```
hekkhekk@hekkhekk:~$ sudo iptables -L -n -v
Chain INPUT (policy DROP 66 packets, 3804 bytes)
pkts bytes target prot opt in out source destination
101 6296 f2b-sshd tcp -- * * 0.0.0.0/0 0.0.0.0/0 multiport dports 22
62953 18M ufw-before-logging-input all -- * * 0.0.0.0/0 0.0.0.0/0
62953 18M ufw-after-input all -- * * 0.0.0.0/0 0.0.0.0/0
69 3944 ufw-after-logging-input all -- * * 0.0.0.0/0 0.0.0.0/0
69 3944 ufw-reject-input all -- * * 0.0.0.0/0 0.0.0.0/0
69 3944 ufw-track-input all -- * * 0.0.0.0/0 0.0.0.0/0

Chain FORWARD (policy DROP 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination
0 0 ufw-before-logging-forward all -- * * 0.0.0.0/0 0.0.0.0/0
0 0 ufw-before-forward all -- * * 0.0.0.0/0 0.0.0.0/0
0 0 ufw-after-forward all -- * * 0.0.0.0/0 0.0.0.0/0
0 0 ufw-after-logging-forward all -- * * 0.0.0.0/0 0.0.0.0/0
0 0 ufw-reject-forward all -- * * 0.0.0.0/0 0.0.0.0/0
0 0 ufw-track-forward all -- * * 0.0.0.0/0 0.0.0.0/0

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination
79894 31M ufw-before-logging-output all -- * * 0.0.0.0/0 0.0.0.0/0
79894 31M ufw-before-output all -- * * 0.0.0.0/0 0.0.0.0/0
15271 924K ufw-after-output all -- * * 0.0.0.0/0 0.0.0.0/0
15271 924K ufw-after-logging-output all -- * * 0.0.0.0/0 0.0.0.0/0
15271 924K ufw-reject-output all -- * * 0.0.0.0/0 0.0.0.0/0
15271 924K ufw-track-output all -- * * 0.0.0.0/0 0.0.0.0/0

Chain f2b-sshd (1 references)
pkts bytes target prot opt in out source destination
101 6296 RETURN all -- * * 0.0.0.0/0 0.0.0.0/0

Chain ufw-after-forward (1 references)
pkts bytes target prot opt in out source destination

Chain ufw-after-input (1 references)
pkts bytes target prot opt in out source destination
0 0 ufw-skip-to-policy-input udp -- * * 0.0.0.0/0 0.0.0.0/0 udp dpt:137
0 0 ufw-skip-to-policy-input udp -- * * 0.0.0.0/0 0.0.0.0/0 udp dpt:138
```

UFW is a front-end tool that simplifies firewall configuration by managing iptables rules.

Intrusion Detection System (IDS)

Intrusion is when an attacker gets unauthorized access to a device, network, or system. Cyber criminals use advanced techniques to sneak into organizations without being detected.

Intrusion Detection System (IDS) observes network traffic for malicious transactions and sends immediate alerts when it is observed. It is software that checks a network or system for malicious activities or policy violations. Each illegal activity or violation is often recorded either centrally using an SIEM system or notified to an administration. IDS monitors a network or system for malicious activity and protects a computer network from unauthorized access from users, including perhaps insiders. The intrusion detector learning task is to build a predictive model (i.e. a classifier) capable of distinguishing between 'bad connections' (intrusion/attacks) and 'good (normal) connections'.

- **Address Spoofing:** Hiding the source of an attack by using fake or unsecured proxy servers making it hard to identify the attacker.
- **Fragmentation:** Sending data in small pieces to slip past detection systems.
- **Pattern Evasion:** Changing attack methods to avoid detection by IDS systems that look for specific patterns.
- **Coordinated Attack:** Using multiple attackers or ports to scan a network, confusing the IDS and making it hard to see what is happening.

Firewall (UFW)	IDS (Suricata/Snort)
Blocks traffic	Monitors traffic
Rule = allow/deny	Rule = detect patterns
Preventive	Detective

IDS = “security camera” — Firewall = “security guard”

An Intrusion Prevention System (IPS) actively monitors network and system traffic to detect and block malicious activity in real time. It extends traditional IDS capabilities by not only identifying threats but also automatically preventing attacks before they cause damage.

Snort and Suricata are both open-source Intrusion Detection Systems used to monitor network traffic and detect malicious activities. Snort is a widely used, traditional IDS that operates in a single-threaded manner, making it suitable for small-scale environments and beginners. In contrast, Suricata is a modern IDS that supports multi-threading, enabling it to handle high-speed network traffic efficiently. Additionally, Suricata provides advanced protocol analysis and better logging capabilities. While Snort is easier to set up and learn, Suricata is preferred in enterprise environments requiring high performance and deeper inspection.

Snort :

Configuring snort

Please use the CIDR form - for example, 192.168.1.0/24 for a block of 256 addresses or 192.168.1.42/32 for just one. Multiple values should be comma-separated (without spaces).

You can leave this value empty and configure HOME_NET in /etc/snort/snort.conf instead. This is useful if you are using Snort in a system which frequently changes network and does not have a static IP address assigned.

Please note that if Snort is configured to use multiple interfaces, it will use this value as the HOME_NET definition for all of them.

Address range for the local network:

192.168.0.0/16

<Ok>

10.0.0.0/24 is correct


```

PS C:\Users\Lenovo> ip a
ip : The term 'ip' is not recognized as the name of a cmdlet, function,
script file, or operable program. Check the spelling of the name, or if a
path was included, verify that the path is correct and try again.
At line:1 char:1
+ ip a
+ ~~~
+ CategoryInfo          : ObjectNotFound: (ip:String) [], CommandNotFo
undException
+ FullyQualifiedErrorId : CommandNotFoundException

```

In windows ipconfig- /16

192.168.0.0 → 192.168.255.255

- Works without worrying about exact network
- Covers all 192.168.x.x ranges
- Too broad
- Detects traffic that may not belong to your actual network
- More **noise / unnecessary alerts**

```

hekhek@hekhek:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:22:48:05:43:fc brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.4/24 metric 100 brd 10.0.0.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::222:48ff:fe05:43fc/64 scope link
        valid_lft forever preferred_lft forever
3: enP64289s1: <BROADCAST,MULTICAST,SLAVE,UP,LOWER_UP> mtu 1500 qdisc mq master eth0 state UP group default qlen 1000
    link/ether 00:22:48:05:43:fc brd ff:ff:ff:ff:ff:ff
    altname enP64289p0s2
    inet6 fe80::222:48ff:fe05:43fc/64 scope link
        valid_lft forever preferred_lft forever

```

10.0.0.0/24

How? "Show all network interfaces and their IP addresses" : 1: lo 2: eth0 3: enP64289s1

- A network interface is a point through which a system connects to a network (physical or virtual).
- Each interface has a unique name (e.g., eth0, wlan0, lo).
- The loopback interface (lo) is used for internal communication within the system (127.0.0.1).
- Physical or virtual interfaces like eth0 or wlan0 connect the system to external networks.
- Each interface can have an IP address assigned to it.
- An IP address uniquely identifies a device in a network.
- The subnet (e.g., /24) defines the range of IP addresses in the network.
- The broadcast address is used to send data to all devices in the same network.

- MTU (Maximum Transmission Unit) defines the maximum size of a packet that can be transmitted.
- Interface states like UP or DOWN indicate whether the interface is active.
- Flags like BROADCAST and MULTICAST indicate supported communication types.
- The interface is what tools like Snort listen on to capture network traffic.
- Correct interface selection is essential for accurate monitoring and detection.

What do the Linux interface names mean?

- eth0
- eth1
- wlan0

My current assumption is that when we are connected to the Internet via LAN cable it's eth0 or eth1 and when we are connected with internet via WiFi it's wlan0.

The names however can be set/chosen by the user or the operating system that you are using. `eth0` and `eth1` is used because it's more intuitive than choosing an arbitrary name because "LAN cable" connection, like you said is Ethernet (hence the `eth` in `eth0`, `eth1`). Similarly when you connect to WiFi, it's "WirelessLAN" (hence the `wlan` in `wlan0`).

Networking Commands		
arp	ipcs	route
curl	iptables	rsync
host	iptables-save	scp
hostid	iwconfig	ssh
hostname	nc (netcat)	tracepath
hostnamectl	netstat	traceroute
ifconfig	nmcli	vnstat
iftop	nslookup	wget
ip	ping	-
ipcrm	rcp	-



Networking commands in Linux

- A virtual machine has its own virtual network interface, commonly named eth0.
- The IP address (e.g., 10.0.0.4) belongs to the virtual machine, not the host system.
- The virtual network is created and managed by the hypervisor (VirtualBox, VMware, etc.). A hypervisor (or Virtual Machine Monitor, VMM) is software that lets multiple operating systems run on a single physical machine.
- The subnet (/24) defines the reachable address range within that virtual network (10.0.0.0–10.0.0.255).
- The interface eth0 is responsible for sending and receiving all network traffic of the virtual machine.
- Network modes determine connectivity behavior: NAT, Bridged, or Host-only.
- In NAT mode, the virtual machine accesses external networks through the host.
- In Bridged mode, the virtual machine appears as an independent device on the same network as the host.
- Snort running inside the virtual machine monitors traffic only on the VM's interface (eth0).
- Traffic that does not pass through eth0 will not be captured or analyzed by Snort.
- Network activity generated within the virtual machine (e.g., ping, SSH) can be detected by Snort.

<https://www.virtualbox.org/manual/ch06.html>

sudo nano /etc/snort/snort.conf

```
# Setup the network addresses you are protecting
#
# Note to Debian users: this value is overridden when starting
# up the Snort daemon through the init.d script by the
# value of DEBIAN_SNORT_HOME_NET s defined in the
# /etc/snort/snort.debian.conf configuration file
#
ipvar HOME_NET 10.0.0.0/24

# Set up the external network addresses. Leave as "any" in most situations
ipvar EXTERNAL_NET any
# If HOME_NET is defined as something other than "any", alternative, you can
# use this definition if you do not want to detect attacks from your intern>
# IP addresses:
#ipvar EXTERNAL_NET !$HOME_NET
```

Which network to protect? 10.0.0.0/24 ; why?

DHCP is a network protocol that automatically gives devices an IP address and other network settings so they can connect and communicate on a network without manual configuration.

- Assigns a unique IP address to each device.
- Provides the subnet mask to define the local network range.
- Configures the default gateway for communication outside the local network.
- Supplies DNS server addresses for domain name resolution.

EXTERNAL_NET = everything outside HOME_NET (attackers can come from anywhere)

etc/snort/rules/local.rules

alert icmp any any -> \$HOME_NET any (msg:"Ping Detected"; sid:1000001; rev:1;)

[action] [protocol] [src_ip] [src_port] -> [dst_ip] [dst_port] (options)

Options:

- **alert** → show alert
- **log** → log only
- **pass** → ignore

Protocol : icmp/tcp/udp/ip(all)

sudo snort -A console -q -c /etc/snort/snort.conf -i eth0 [Running snort]

```
hekhek@hekhek:~$ snort
Running in packet dump mode

--== Initializing Snort ==--
Initializing Output Plugins!
pcap DAQ configured to passive.
Acquiring network traffic from "eth0".
ERROR: Can't start DAQ (-1) - socket: Operation not permitted!
Fatal Error, Quitting..
```

```
hekhek@hekhek:~$ sudo snort
Running in packet dump mode

--== Initializing Snort ==--
Initializing Output Plugins!
pcap DAQ configured to passive.
Acquiring network traffic from "eth0".
Decoding Ethernet

--== Initialization Complete ==--

    _ _ _ _ _
   /  _  _  \
  /  _  _  \
 /  _  _  \
/  _  _  \
o"  )~
'   '

-*> Snort! <*-
Version 2.9.15.1 GRE (Build 15125)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2019 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.10.1 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Commencing packet processing (pid=28243)
03/29-12:16:21.186809 10.0.0.4:2222 -> 106.76.189.159:38159
TCP TTL:64 TOS:0x10 ID:11132 IpLen:20 DgmLen:100 DF
***AP*** Seq: 0x111B20CD Ack: 0x25A7385C Win: 0x2BB TcpLen: 20
```

When sudo snort - Initialization completed and within few seconds many WARNINGS telling that no preprocessors configured for policy 0 was mentioned

By default, without options, Snort:

- tries to read default config
- may run in packet logging mode

-i = interface selection option [-i <interface_name>]

-i eth0 -> Attach Snort to interface named eth0

Internally :

- **Snort binds to eth0 using libpcap** [a system-independent interface for user-level packet capture. libpcap provides a portable framework for low-level network monitoring. Applications include network statistics collection, security monitoring, network debugging, etc. Since almost every system vendor provides a different interface for packet capture, and since we've developed several tools that require this functionality, we've created this system-independent API to ease in porting and to alleviate the need for several system-dependent packet capture modules in each application.]
- **Starts capturing packets flowing through that interface**

-c = configuration file option {Load configuration from this file instead of default}

-A = alert mode option , console = argument to that option

-A <mode> [modes : console,fast,full,etc..] -q = quiet mode

ping 8.8.8.8 : **Ping (Packet Internet Groper)** is a basic Internet command used in computer network administration to test and verify the availability of a destination IP address. It helps users confirm whether the computer they are trying to reach is operational. Ping works by measuring the communication latency which is the time taken for data to travel between two devices or across a network. As communication latency decreases, the effectiveness of communication improves and hence makes it an essential tool for network troubleshooting. It can be used with any operating system (OS) that supports networking.

- Ping uses a combination of Internet Control Message Protocol (ICMP), Echo Request and response messages.
- When a network administrator inputs a ping command on the command prompt, an ICMP Echo Request packet is sent to the specified IP Address.
- When the target host receives the echo request and if available or online, it responds with an ICMP Echo Reply packet.

Snort listens on eth0, captures the incoming ICMP reply from 8.8.8.8, matches it with the rule targeting HOME_NET, and generates an alert.

User and Permission Management